

Gin Web Framework

Tài liệu tham khảo đầy đủ — dựa trên gin-gonic.com/en/docs

Phiên bản: Gin v1.x | Yêu cầu: Go 1.25+ | github.com/gin-gonic/gin

Nội dung: Quickstart · Routing · Binding & Validation · Rendering · Middleware · Logging · Server Config · Testing

1. Cài đặt & Khởi tạo

Yêu cầu

- Go phiên bản 1.25 trở lên (kiểm tra bằng `go version`)
- Go phải có trong PATH và chạy được từ terminal

Khởi tạo project mới

```
mkdir gin-quickstart && cd gin-quickstart // tạo thư mục project
go mod init gin-quickstart // khởi tạo Go module
go get -u github.com/gin-gonic/gin // cài Gin
```

Hello World — Ứng dụng đầu tiên

Tạo file `main.go` với nội dung sau:

```
package main

import (
    "net/http"
    "github.com/gin-gonic/gin"
)

func main() {
    router := gin.Default() // tạo router có sẵn Logger + Recovery
    middleware

    router.GET("/ping", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{
            "message": "pong",
        })
    })

    router.Run() // lắng nghe trên 0.0.0.0:8080 mặc định
    // router.Run(":3000") - để chỉ định port cụ thể
}
```

Chạy server:

```
go run main.go
# Truy cập: http://localhost:8080/ping
# Kết quả: {"message":"pong"}
```

Sử dụng với net/http

Dùng hằng số HTTP status code từ net/http thay vì số nguyên thô:

```
router.GET("/ping", func(c *gin.Context) {
    c.JSON(http.StatusOK, gin.H{"message": "pong"})    // http.StatusOK = 200
})
```

Các chế độ hoạt động

Mode	Mô tả
gin.DebugMode	Mặc định — in log debug, route registration ra console
gin.ReleaseMode	Production — tắt debug log, hiệu năng tốt hơn, nên dùng khi deploy
gin.TestMode	Dùng trong unit test — tắt output không cần thiết

```
gin.SetMode(gin.ReleaseMode)    // đặt TRƯỚC khi tạo router
router := gin.Default()
// hoặc dùng biến môi trường
// GIN_MODE=release go run main.go
```

❑ Luôn set ReleaseMode trước khi deploy production để tránh lộ thông tin debug.

gin.Default() vs gin.New()

Hàm	Middleware đi kèm
gin.Default()	Có sẵn gin.Logger() + gin.Recovery() — khuyên dùng
gin.New()	Router rỗng, không có middleware nào — tự thêm thủ công

2. Routing

HTTP Methods

Gin cung cấp method tương ứng với mỗi HTTP verb. Mỗi method chỉ nhận request đúng loại đó:

```
router.GET("/users", getUsers)    // Lấy tài nguyên
router.POST("/users", createUser) // Tạo mới tài nguyên
```

```

router.PUT("/users/:id", updateUser)    // Thay thế toàn bộ tài nguyên
router.PATCH("/users/:id", patchUser)   // Cập nhật một phần tài nguyên
router.DELETE("/users/:id", deleteUser) // Xóa tài nguyên
router.HEAD("/users", headUser)         // Như GET nhưng không có body
router.OPTIONS("/users", optionsUser)   // Mô tả các tùy chọn giao tiếp

router.Any("/path", handler)            // Nhận tất cả HTTP methods
router.Handle("GET", "/path", handler)  // Dạng tổng quát – tự chỉ định
method

```

Method	REST Usage	Ghi chú
GET	Lấy tài nguyên	Idempotent, safe
POST	Tạo mới	Không idempotent
PUT	Thay thế hoàn toàn	Idempotent
PATCH	Cập nhật một phần	Không bắt buộc idempotent
DELETE	Xóa tài nguyên	Idempotent
HEAD	Headers only	Dùng để kiểm tra tồn tại, không có body
OPTIONS	CORS preflight	Mô tả khả năng của server

Parameters in Path

Dùng `:param` cho tham số bắt buộc, `*param` để bắt tất cả phần còn lại của path:

```

// :id - bắt buộc, không match /users/
router.GET("/users/:id", func(c *gin.Context) {
    id := c.Param("id")    // vd: /users/42 → id = "42"
    c.JSON(http.StatusOK, gin.H{"id": id})
})

// *filepath - bắt tất cả, có dấu / đầu tiên
router.GET("/static/*filepath", func(c *gin.Context) {
    path := c.Param("filepath")    // vd: /static/js/app.js → /js/app.js
})

```

Query String Parameters

```

// GET /search?q=gin&page=2&tag=web&tag=api
q := c.Query("q")    // trả "" nếu không có
q := c.DefaultQuery("q", "all")    // trả giá trị mặc định nếu không có
tags := c.QueryArray("tag")    // lấy nhiều giá trị: ["web", "api"]
filters := c.QueryMap("filter")    // map: ?filter[name]=gin&filter[v]=1

```

Query and Post Form

Có thể đọc cả query string lẫn form data trong cùng một handler:

```
// POST /post?id=123&page=1 với body: name=gin&message=hello
func(c *gin.Context) {
    id      := c.Query("id")    // từ query string: 123
    page    := c.DefaultQuery("page", "0") // từ query string: 1
    name    := c.PostForm("name") // từ form body: gin
    message := c.PostForm("message") // từ form body: hello
    c.JSON(200, gin.H{"id":id,"page":page,"name":name,"msg":message})
}
```

Map as Querystring hoặc PostForm

```
// ?ids[a]=1234&ids[b]=hello
ids := c.QueryMap("ids") // map[a:1234 b:hello]

// Form: names[first]=thinkerou&names[second]=tianou
names := c.PostFormMap("names") // map[first:thinkerou second:tianou]
```

Multipart / Urlencoded Form

```
func(c *gin.Context) {
    message := c.PostForm("message")
    nick    := c.DefaultPostForm("nick", "anonymous")
    c.JSON(200, gin.H{"status":"posted","message":message,"nick":nick})
}

// Test bằng curl:
// curl -X POST http://localhost:8080/form \
// -F 'message=hello' -F 'nick=gin'
```

Upload Files

Upload 1 file

```
router.POST("/upload", func(c *gin.Context) {
    file, err := c.FormFile("file") // lấy file từ form field "file"
    if err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    dst := "." + file.Filename
    c.SaveUploadedFile(file, dst) // lưu vào disk
    c.JSON(200, gin.H{"uploaded": file.Filename})
})
```

Upload nhiều files

```
router.POST("/upload", func(c *gin.Context) {
    form, _ := c.MultipartForm()
```

```

files := form.File["upload[]"] // lấy slice file từ field upload[]
for _, file := range files {
    c.SaveUploadedFile(file, "."+file.Filename)
}
c.JSON(200, gin.H{"uploaded": len(files)})
})

```

Giới hạn dung lượng upload

```

router.MaxMultipartMemory = 8 << 20 // giới hạn 8 MiB (mặc định 32 MiB)

```

Grouping Routes

Group giúp tổ chức routes theo prefix và áp dụng middleware cho một nhóm cụ thể:

```

v1 := router.Group("/v1")
{
    v1.POST("/login", loginEndpoint) // → POST /v1/login
    v1.POST("/submit", submitEndpoint) // → POST /v1/submit
    v1.POST("/read", readEndpoint) // → POST /v1/read
}

v2 := router.Group("/v2")
{
    v2.POST("/login", loginEndpoint) // → POST /v2/login
    v2.POST("/submit", submitEndpoint) // → POST /v2/submit
    v2.POST("/read", readEndpoint) // → POST /v2/read
}

```

Redirects

```

// HTTP redirect
router.GET("/redirect", func(c *gin.Context) {
    c.Redirect(http.StatusMovedPermanently, "https://google.com") // 301
})

// Router redirect (internal)
router.GET("/old", func(c *gin.Context) {
    c.Request.URL.Path = "/new"
    router.HandleContext(c) // chuyển hướng nội bộ
})
router.GET("/new", func(c *gin.Context) {
    c.JSON(200, gin.H{"message": "new route"})
})

```

3. Binding & Validation

Tổng quan

Gin dùng `go-playground/validator/v10` để validate. Có hai nhóm method binding:

Nhóm	Methods	Hành vi khi lỗi
Must Bind	Bind, BindJSON, BindXML, BindQuery, BindYAML	Tự abort request với status 400, set Content-Type: text/plain
Should Bind	ShouldBind, ShouldBindJSON, ShouldBindXML, ShouldBindQuery, ShouldBindYAML	Trả error — developer tự xử lý; kiểm soát tốt hơn

❑ Nên dùng `ShouldBind*` để kiểm soát lỗi tốt hơn. `MustBind*` có thể gây warning nếu status đã được set.

Model Binding và Validation

Định nghĩa struct với binding tags, rồi bind từ JSON, XML, hoặc Form:

```
type Login struct {
    User      string `form:"user" json:"user" xml:"user" binding:"required"`
    Password string `form:"password" json:"password" xml:"password"
binding:"required"`
}

func main() {
    router := gin.Default()

    // Bind từ JSON body
    router.POST("/loginJSON", func(c *gin.Context) {
        var json Login
        if err := c.ShouldBindJSON(&json); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
            return
        }
        c.JSON(http.StatusOK, gin.H{"status": "logged in"})
    })

    // Bind từ XML body
    router.POST("/loginXML", func(c *gin.Context) {
        var xml Login
        if err := c.ShouldBindXML(&xml); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
            return
        }
        c.JSON(http.StatusOK, gin.H{"status": "logged in"})
    })

    // Bind từ HTML Form (tự suy ra binder từ Content-Type)
    router.POST("/loginForm", func(c *gin.Context) {
        var form Login
        if err := c.ShouldBind(&form); err != nil {
```

```

        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }
    c.JSON(http.StatusOK, gin.H{"status": "logged in"})
})

router.Run(":8080")
}

```

Bind URI

```

type URI struct {
    ID    uint   `uri:"id" binding:"required"`
    Name  string `uri:"name" binding:"required"`
}

router.GET("/:name/:id", func(c *gin.Context) {
    var uri URI
    if err := c.ShouldBindUri(&uri); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    c.JSON(200, gin.H{"name": uri.Name, "id": uri.ID})
})

```

Bind Header

```

type Header struct {
    RateLimit int `header:"Rate-Limit" binding:"required"`
}

router.GET("/header", func(c *gin.Context) {
    var h Header
    if err := c.ShouldBindHeader(&h); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }
    c.JSON(200, gin.H{"rate_limit": h.RateLimit})
})

```

Bind Default Values

default tag cho phép đặt giá trị mặc định khi field không được gửi:

```

type Params struct {
    Name  string `form:"name" default:"unknown"`
    Page  int    `form:"page" default:"1"`
    Size  int    `form:"size" default:"20"`
}

```

```
// GET /list → Name="unknown", Page=1, Size=20
// GET /list?name=gin&page=3 → Name="gin", Page=3, Size=20
```

Custom Validators

```
var bookableValidator validator.Func = func(fl validator.FieldLevel) bool {
    date, ok := fl.Field().Interface().(time.Time)
    if ok {
        today := time.Now()
        return today.Before(date)    // ngày phải trong tương lai
    }
    return false
}

// Đăng ký validator
if v, ok := binding.Validator.Engine().(*validator.Validate); ok {
    v.RegisterValidation("bookabledat", bookableValidator)
}

type Booking struct {
    CheckIn time.Time `form:"check_in" binding:"required,bookabledat"`
    CheckOut time.Time `form:"check_out" binding:"required,gtefield=CheckIn"`
}
```

Bind HTML Checkboxes

```
type myForm struct {
    Colors []string `form:"colors[]"`
}

router.POST("/color", func(c *gin.Context) {
    var form myForm
    c.ShouldBind(&form)    // bind từ form-data
    c.JSON(200, gin.H{"colors": form.Colors})
})
```

Try to Bind Body into Different Structs

`c.ShouldBind` đọc và tiêu thụ body. Dùng `c.ShouldBindBodyWith` để đọc nhiều lần:

```
func SomeHandler(c *gin.Context) {
    objA := formA{}
    if err := c.ShouldBindBodyWith(&objA, binding.JSON); err == nil {
        c.String(200, "the body was %s", objA.Foo)
    }

    objB := formB{}
    // body đã được cache sau lần gọi đầu
    if err := c.ShouldBindBodyWith(&objB, binding.JSON); err == nil {
        c.String(200, "the body was %s", objB.Bar)
    }
}
```

```
}  
}
```

□ ShouldBindBodyWith chỉ cần thiết khi bind body nhiều lần. Với 1 lần bind, dùng ShouldBindJSON/ShouldBindXML thông thường.

4. Rendering

JSON / XML / YAML / ProtoBuf

Gin hỗ trợ nhiều định dạng response. Mỗi method tự set Content-Type phù hợp:

```
// JSON — dùng gin.H (shortcut cho map[string]interface{})  
router.GET("/json", func(c *gin.Context) {  
    c.JSON(http.StatusOK, gin.H{"message": "hey", "status": http.StatusOK})  
})  
  
// JSON với struct — field json tag đổi tên  
var msg struct {  
    Name    string `json:"user"`  
    Message string  
    Number  int  
}  
// Output: {"user":"Lena","Message":"hey","Number":123}  
c.JSON(http.StatusOK, msg)  
  
// XML  
c.XML(http.StatusOK, gin.H{"message": "hey"})  
  
// YAML  
c.YAML(http.StatusOK, gin.H{"message": "hey"})  
  
// ProtoBuf — cần .proto schema  
c.ProtoBuf(http.StatusOK, protoData) // binary response
```

Các biến thể của JSON

Method	Mô tả
<code>c.JSON()</code>	JSON chuẩn — escape HTML characters (<, >, &)
<code>c.IndentedJSON()</code>	JSON có indent — dễ đọc, dùng khi develop
<code>c.PureJSON()</code>	JSON không escape HTML — trả nguyên ký tự như <, >
<code>c.SecureJSON()</code>	Thêm prefix while(1); chống JSON hijacking
<code>c.AsciiJSON()</code>	Escape non-ASCII sang luXXXX — safe cho mọi encoding
<code>c.JSONP()</code>	JSONP — bọc trong callback function cho cross-domain

```
// PureJSON - không escape HTML
c.PureJSON(200, gin.H{"html": "<b>Hello</b>"})
// Output: {"html":"<b>Hello</b>"} (KHÔNG phải \u003c)

// SecureJSON
c.SecureJSON(200, []string{"one","two"})
// Output: while(1);["one","two"]

// JSONP - GET /jsonp?callback=x
c.JSONP(200, gin.H{"foo": "bar"})
// Output: x({"foo":"bar"})
```

Serving Static Files

```
// Map URL prefix → thư mục trên disk
router.Static("/assets", "./assets") // phục vụ toàn bộ thư mục
router.StaticFS("/more_static", http.Dir("my_file_system")) // dùng
http.FileSystem
router.StaticFile("/favicon.ico", "./resources/favicon.ico") // phục vụ 1
file cụ thể

// Serving data từ reader (ví dụ: remote file)
response, _ := http.Get("https://example.com/image.png")
c.DataFromReader(http.StatusOK, response.ContentLength,
    "image/png", response.Body, nil)
```

HTML Rendering

```
// Load templates
router.LoadHTMLGlob("templates/*") // load tất cả file trong thư mục
router.LoadHTMLFiles("templates/index.html", "templates/login.html") // load
file cụ thể

router.GET("/", func(c *gin.Context) {
    c.HTML(http.StatusOK, "index.html", gin.H{
        "title": "Main website",
    })
})

// Custom template functions
router.SetFuncMap(template.FuncMap{
    "formatAsDate": formatAsDate,
}) // phải gọi TRƯỚC LoadHTMLGlob
router.LoadHTMLGlob("templates/**/*.html")
```

Multitemplate

Dùng khi cần nhiều thư mục template hoặc cần phân biệt template có cùng tên:

```
import "github.com/gin-contrib/multitemplate"
```

```
func createMyRender() multitemplate.Renderer {
    r := multitemplate.NewRenderer()
    r.AddFromFiles("index", "base.html", "index.html")
    r.AddFromFiles("article", "base.html", "article.html")
    return r
}

router.HTMLRender = createMyRender()
router.GET("/", func(c *gin.Context) {
    c.HTML(200, "index", gin.H{"title": "Home"})
})
```

5. Middleware

Cách hoạt động của Middleware

Middleware là hàm trả về `gin.HandlerFunc`. Nó chạy trước và/hoặc sau handler chính, chia làm 2 pha bởi lời gọi `c.Next()`:

Pha	Mô tả
Trước <code>c.Next()</code>	Code chạy trước khi request tới handler. Dùng cho: ghi thời gian bắt đầu, validate token, set context value (<code>c.Set()</code>)
<code>c.Next()</code>	Gọi handler tiếp theo trong chain. Execution tạm dừng cho đến khi tất cả downstream handler hoàn thành
Sau <code>c.Next()</code>	Code chạy sau khi handler xong. Dùng cho: logging response status, tính latency, cleanup resources

□ Gọi `c.Abort()` thay vì `c.Next()` để dừng chain (ví dụ: authentication fail). Kết hợp với `c.AbortWithStatusJSON()` để trả lỗi luôn.

Custom Middleware

```
func Logger() gin.HandlerFunc {
    return func(c *gin.Context) {
        t := time.Now()

        // Set giá trị vào context
        c.Set("example", "12345")

        c.Next()    // gọi handler tiếp theo

        // Sau khi handler chạy xong
        latency := time.Since(t)
        log.Print(latency)
        status := c.Writer.Status()
    }
}
```

```

        log.Println(status)
    }
}

func main() {
    r := gin.New()
    r.Use(Logger())    // đăng ký middleware global

    r.GET("/test", func(c *gin.Context) {
        example := c.MustGet("example").(string)    // lấy giá trị từ context
        log.Println(example)    // in "12345"
    })
    r.Run()
}

```

Đăng ký Middleware

```

// Global - cho tất cả routes
r := gin.New()
r.Use(gin.Logger())
r.Use(gin.Recovery())

// Cho route group
authorized := r.Group("/")
authorized.Use(AuthRequired())
{
    authorized.POST("/login", loginEndpoint)
    authorized.POST("/submit", submitEndpoint)
}

// Trực tiếp trên route đơn lẻ
r.GET("/benchmark", MyBenchLogger(), benchEndpoint)

```

Using BasicAuth Middleware

```

var secrets = gin.Accounts{
    "foo":    "bar",    // user: foo, pass: bar
    "austin": "1234",
    "lena":   "hello2",
}

authorized := r.Group("/admin", gin.BasicAuth(secrets))
authorized.GET("/secrets", func(c *gin.Context) {
    user := c.MustGet(gin.AuthUserKey).(string)    // lấy username đã auth
    if secret, ok := secrets[user]; ok {
        c.JSON(200, gin.H{"user": user, "secret": secret})
    } else {
        c.JSON(200, gin.H{"user": user, "secret": "NO SECRET :("})
    }
})

```

Error Handling Middleware

```
func ErrorHandler() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Next()    // chạy handler trước

        // Xử lý errors sau khi handler kết thúc
        for _, err := range c.Errors {
            log.Println(err)
        }
        if len(c.Errors) > 0 {
            c.JSON(-1, c.Errors)    // -1 = giữ nguyên status đã set
        }
    }
}

router.GET("/test", func(c *gin.Context) {
    c.Error(errors.New("something went wrong"))    // thêm lỗi vào context
    c.AbortWithStatus(500)
})
```

Goroutines inside Middleware

Khi muốn dùng `*gin.Context` trong goroutine, phải dùng `c.Copy()` để tạo bản sao — không được dùng con trỏ gốc:

```
r.GET("/long_async", func(c *gin.Context) {
    cCp := c.Copy()    // tạo bản sao context
    go func() {
        time.Sleep(5 * time.Second)
        log.Println("Done! In path " + cCp.Request.URL.Path)    // dùng bản sao
    }()
})

// KHÔNG làm thế này:
// r.GET("/wrong", func(c *gin.Context) {
//     go func() { log.Println(c.Request.URL.Path) }() ← RACE CONDITION
// })
```

6. Logging

Ghi log ra file

`gin.DefaultWriter` mặc định trỏ vào `os.Stdout`. Thay đổi trước khi tạo router:

```
gin.DisableConsoleColor()    // tắt màu khi ghi vào file

f, _ := os.Create("gin.log")
gin.DefaultWriter = io.MultiWriter(f)    // chỉ ghi vào file
```

```
// hoặc ghi vào cả file lẫn console:
// gin.DefaultWriter = io.MultiWriter(f, os.Stdout)

router := gin.Default()
```

Log rotation với lumberjack

`os.Create` truncate file mỗi lần restart. Dùng `lumberjack` để rotate theo size/thời gian:

```
import "gopkg.in/natefinch/lumberjack.v2"

gin.DisableConsoleColor()
gin.DefaultWriter = &lumberjack.Logger{
    Filename:   "gin.log",
    MaxSize:    100, // megabytes
    MaxBackups: 3,   // số file backup giữ lại
    MaxAge:     28,  // ngày
}
```

Custom Log Format

```
router.Use(gin.LoggerWithFormatter(func(param gin.LogFormatterParams) string {
    return fmt.Sprintf("%s - [%s] \"%s %s %s %d %s \"%s\" %s\"\\n",
        param.ClientIP,
        param.TimeStamp.Format(time.RFC1123),
        param.Method,
        param.Path,
        param.Request.Proto,
        param.StatusCode,
        param.Latency,
        param.Request.UserAgent(),
        param.ErrorMessage,
    )
}))
```

Controlling Log Output Coloring

```
// Bật màu (mặc định nếu terminal hỗ trợ)
gin.ForceConsoleColor()

// Tắt màu
gin.DisableConsoleColor()
```

Skip Logging cho một số routes

```
router.Use(gin.LoggerWithConfig(gin.LoggerConfig{
    SkipPaths: []string{"/health", "/metrics"}, // không log 2 route này
})
```

```
)))
```

7. Server Configuration

Custom HTTP Server

Dùng `http.Server` trực tiếp thay vì `router.Run()` để kiểm soát timeout và các tham số chi tiết:

```
s := &http.Server{
    Addr:           ":8080",
    Handler:        router,
    ReadTimeout:    10 * time.Second,
    WriteTimeout:   10 * time.Second,
    MaxHeaderBytes: 1 << 20,    // 1 MiB
}
s.ListenAndServe()
```

Graceful Restart / Stop

Graceful shutdown cho phép hoàn thành các request đang xử lý trước khi dừng server — tránh mất request khi deploy:

```
router := gin.Default()
router.GET("/", func(c *gin.Context) {
    time.Sleep(5 * time.Second) // giả lập request chạy lâu
    c.String(http.StatusOK, "Welcome Gin Server")
})

srv := &http.Server{Addr: ":8080", Handler: router.Handler()}

go func() { // chạy server trong goroutine
    if err := srv.ListenAndServe(); err != nil &&
        err != http.ErrServerClosed {
        log.Fatalf("listen: %s\n", err)
    }
}()

// Chờ tín hiệu dừng (Ctrl+C hoặc kill)
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit // block cho đến khi nhận signal
log.Println("Shutdown Server ...")

ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
if err := srv.Shutdown(ctx); err != nil {
    log.Fatal("Server Shutdown:", err)
}
log.Println("Server exiting")
```

❑ Kết hợp với load balancer, graceful shutdown cho phép zero-downtime deployment.

HTTPS / TLS

```
// Certificate thủ công
router.RunTLS(":443", "cert.pem", "key.pem")

// Tự động với Let's Encrypt (autotls)
import "github.com/gin-gonic/autotls"
m := autocert.Manager{
    Prompt:      autocert.AcceptTOS,
    HostPolicy:  autocert.HostWhitelist("example1.com", "example2.com"),
    Cache:       autocert.DirCache("/var/www/.cache"),
}
log.Fatal(autotls.RunWithManager(r, &m))
```

Run Multiple Services

```
var wg sync.WaitGroup
wg.Add(2)

go func() { // server 1
    defer wg.Done()
    server01 := &http.Server{Addr: ":8080", Handler: router01}
    server01.ListenAndServe()
}()

go func() { // server 2
    defer wg.Done()
    server02 := &http.Server{Addr: ":8081", Handler: router02}
    server02.ListenAndServe()
}()

wg.Wait() // chờ cả 2 server
```

Cookie

```
// Set cookie
router.GET("/cookie", func(c *gin.Context) {
    cookie, err := c.Cookie("gin_cookie") // đọc cookie
    if err != nil {
        cookie = "NotSet"
        c.SetCookie("gin_cookie", "test", 3600,
            "/", "localhost", false, true) // name, value, maxAge, path,
        domain, secure, httpOnly
    }
    fmt.Printf("Cookie value: %s \n", cookie)
})
```

Trusted Proxies

Gin tin tưởng tất cả proxy mặc định. Trong production, giới hạn trusted proxies để `ClientIP()` chính xác:

```
// Chỉ tin tưởng proxy cụ thể
router.SetTrustedProxies([]string{"192.168.1.0/24"})

// Tắt hoàn toàn trust proxy (direct connection)
router.SetTrustedProxies(nil)

// Sau đó c.ClientIP() sẽ trả IP chính xác
```

HTTP/2 Server Push

```
router.GET("/", func(c *gin.Context) {
    if pusher := c.Writer.Pusher(); pusher != nil {
        if err := pusher.Push("/assets/app.js", nil); err != nil {
            log.Printf("Failed to push: %v", err)
        }
    }
    c.HTML(200, "index.html", nil)
})
```

8. Testing

Cách viết Test cho Gin

`net/http/httptest` là cách được khuyến nghị để test HTTP handler trong Gin:

```
// main.go
type User struct {
    Username string `json:"username"`
    Gender   string `json:"gender"`
}

func setupRouter() *gin.Engine {
    router := gin.Default()
    router.GET("/ping", func(c *gin.Context) {
        c.String(200, "pong")
    })
    return router
}

func postUser(router *gin.Engine) *gin.Engine {
    router.POST("/user/add", func(c *gin.Context) {
        var user User
        c.BindJSON(&user)
        c.JSON(200, user)
    })
}
```

```
    return router
}
```

```
// main_test.go
func TestPingRoute(t *testing.T) {
    router := setupRouter()

    w := httptest.NewRecorder() // ghi lại response
    req, _ := http.NewRequest("GET", "/ping", nil)
    router.ServeHTTP(w, req)

    assert.Equal(t, 200, w.Code)
    assert.Equal(t, "pong", w.Body.String())
}

func TestPostUser(t *testing.T) {
    router := setupRouter()
    router = postUser(router)

    w := httptest.NewRecorder()
    exampleUser := User{Username: "test_name", Gender: "male"}
    userJson, _ := json.Marshal(exampleUser)

    req, _ := http.NewRequest("POST", "/user/add",
        strings.NewReader(string(userJson)))
    router.ServeHTTP(w, req)

    assert.Equal(t, 200, w.Code)
    assert.Equal(t, string(userJson), w.Body.String())
}
```

❑ Luôn gọi `gin.SetMode(gin.TestMode)` ở đầu test file để tắt debug output không cần thiết.

9. Quick Reference

gin.Context — Các method hay dùng nhất

Method	Mô tả
<code>c.Param("name")</code>	Lấy path parameter
<code>c.Query("key")</code>	Lấy query string (trả "" nếu không có)
<code>c.DefaultQuery("k", "v")</code>	Lấy query string với giá trị mặc định
<code>c.PostForm("key")</code>	Lấy form field từ body
<code>c.FormFile("file")</code>	Lấy uploaded file
<code>c.GetHeader("key")</code>	Lấy HTTP header
<code>c.ClientIP()</code>	IP của client (hỗ trợ X-Forwarded-For)
<code>c.ContentType()</code>	Content-Type của request

<code>c.Set("key", val)</code>	Lưu giá trị vào context
<code>c.Get("key")</code>	Lấy giá trị từ context → (any, bool)
<code>c.MustGet("key")</code>	Lấy giá trị hoặc panic nếu không có
<code>c.Next()</code>	Gọi handler tiếp theo trong chain
<code>c.Abort()</code>	Dừng chain middleware/handler
<code>c.AbortWithStatus(403)</code>	Dừng + set HTTP status
<code>c.AbortWithStatusJSON()</code>	Dừng + trả JSON response
<code>c.Copy()</code>	Tạo bản sao context để dùng trong goroutine
<code>c.FullPath()</code>	Pattern của route (vd: /users/:id)
<code>c.IsAborted()</code>	Kiểm tra đã abort chưa

Response Methods

Method	Mô tả
<code>c.JSON(status, obj)</code>	Trả JSON response
<code>c.XML(status, obj)</code>	Trả XML response
<code>c.YAML(status, obj)</code>	Trả YAML response
<code>c.String(status, fmt, ...)</code>	Trả plain text
<code>c.HTML(status, name, data)</code>	Render HTML template
<code>c.File(filepath)</code>	Trả file
<code>c.FileAttachment(path, name)</code>	Trả file với tên download tùy chỉnh
<code>c.Redirect(status, url)</code>	HTTP redirect
<code>c.Status(code)</code>	Chỉ set status, không có body
<code>c.Data(status, ct, data)</code>	Trả raw bytes với content type tùy chỉnh
<code>c.Stream(func(w io.Writer) bool)</code>	Streaming response